问题重述

对于 $n \times n$ 的灯泡矩阵，每次按权重随机点亮一个灯泡，灯泡一旦点亮永不熄灭。
当某一行、列或对角线（共 $m = 2n + 2$ 条）上的 n 个灯泡全部点亮时，对应礼物被获得。
给定礼物集合 S（用位掩码表示），求“第一次集齐 S 中所有礼物”的期望抽奖次数。

关键思路

1. **期望最大值**
   设 T_k 为第 k 条礼物线首次被完成的时间，所求为

$$E\left[\max_{k \in S} T_k\right].$$

2. **容斥原理**
   对任意随机变量 X_i，有

$$E\left[\max_i X_i\right] = \sum_{\emptyset \neq I \subseteq \{1,\ldots,m\}} (-1)^{|I|-1} E\left[\min_{i \in I} X_i\right].$$

3. **化简**
   令 $U_S = \bigcup_{k \in S} C_k$（$C_k$ 为第 k 条线的灯泡集合），则

$$E\left[\max_{k \in S} T_k\right] = E\left[\max_{c \in U_S} X_c\right] = \sum_{\emptyset \neq T \subseteq U_S} (-1)^{|T|-1} \frac{W}{\sum_{c \in T} w_c},$$

   其中 X_c 为灯泡 c 首次被点亮的时间，几何分布 $E[X_c] = 1/p_c$。

动态规划优化

- 直接枚举 $2^{|U_S|}$ 不可行（$|U_S| \leq 49$）。
- 注意到 $\sum_{c \in T} w_c \leq W \leq 1000$，可改为求

$$-W\sum_{k=1}^{W}\frac{f[k]}{k},\quad f[k]=\sum_{\substack{T\subseteq U_S\\ \sum w_c=k}}(-1)^{|T|}.$$

- $f[k]$ 为多项式 $P(z)=\prod_{c\in U_S}(1-z^{w_c})$ 中 z^k 的系数。

预处理所有查询

- 礼物线共 $m=2n+2$ 条，$2^m\le 2^{16}=65536$ 种位掩码可全部预处理。
- DFS 遍历子集，同时维护背包数组 f（长度 $W+1$）。
- 时间复杂度：$O(n\cdot W\cdot 2^m)\approx 7\cdot 1000\cdot 2^{16}\approx 4.6\times 10^8$，可接受。

伪代码

```
dfs(line_idx, mask, f[0..W]):
    if line_idx == m:
    ans[mask] = -W * Σ_{k=1..W} f[k]/k
return
dfs(line_idx+1, mask, f)                // skip line_idx
for bulb in line[line_idx]:
if vis[bulb]++ == 0:                // first time include bulb
for j = W downto w[bulb]:
f[j] -= f[j - w[bulb]]
dfs(line_idx+1, mask | (1<<line_idx), f)
for bulb in line[line_idx]:
if --vis[bulb] == 0:                // rollback
for j = w[bulb]..W:
f[j] += f[j - w[bulb]]
```

样例复杂度

- $n = 7,\ m = 16,\ W = 1000$

$$7 \cdot 1000 \cdot 2^{16} \approx 4.6 \times 10^8.$$

- $n = 6,\ m = 14$ 时约 9.8×10^7，轻松通过。

代码要点

- 使用 `ModInt<1e9+7>` 处理模数。
- 用 Barrett 除法优化逆元。
- `vis` 数组全局共享，回溯时撤销标记。
- 主循环读入 q 次查询，直接输出 `ans[mask]`。

详细分析

好的，我们来详细分析一下这个期望问题，特别是 E[min(S)] 的计算以及 dfs + DP 的实现细节。

1. 问题重述与期望公式
   首先，我们明确一下问题目标：对于给定的一个目标礼物集合 S_{gifts}（由查询 t_i 的二进制位表示），我们需要计算获得所有这些礼物所需的期望抽奖次数。

设 T_k 为获得第 k 种礼物（即第 k 条线上的所有灯泡都亮起）所需的时间。我们要求的是 $E[\max_{k \in S_{gifts}} T_k]$。

进一步，设 X_c 为灯泡 c 亮起所需的时间。那么，获得第 k 种礼物的时间 T_k 实际上是该线上所有灯泡亮起时间的最大值：$T_k = \max_{c \in C_k} X_c$，其中 C_k 是第 k 条线上的灯泡集合。

因此，我们最终要求的是 $E[\max_{k \in S_{gifts}} (\max_{c \in C_k} X_c)]$。 这个表达式可以简化为 $E[\max_{c \in U_{S_{gifts}}} X_c]$，其中 $U_{S_{gifts}} = \bigcup_{k \in S_{gifts}} C_k$ 是所有目标礼物涉及到的所有唯一灯泡的集合。

现在，问题转化为计算 $E[\max_{c \in U_{S_{gifts}}} X_c]$。这是一个典型的“最大值期望”问题，可以使用容斥原理 (Principle of Inclusion-Exclusion, PIE) 来解决。

容斥原理对于最大值期望的公式是：

$$E[\max(X_1, \ldots, X_m)] = \sum_{\emptyset \neq I \subseteq 1,\ldots,m} (-1)^{|I|-1} E[\min_{i \in I} X_i]$$

将此应用于我们的问题，其中 X_i 对应于 $U_{S_{gifts}}$ 中的每个灯泡 c 的亮起时间 X_c：

$$E[\max_{c \in U_{S_{gifts}}} X_c] = \sum_{\emptyset \neq T \subseteq U_{S_{gifts}}} (-1)^{|T|-1} E[\min_{c \in T} X_c]$$

2. 计算 $E[\min_{c \in T} X_c]$
   $E[\min_{c \in T} X_c]$ 表示集合 T 中任意一个灯泡亮起所需的期望抽奖次数。 这是一个经典的几何分布问题。

每次抽奖的机制：每次抽奖，会以概率 $p_{i,j} = \frac{w_{i,j}}{W}$ 选中灯泡 (i, j)。
“成功”的定义：对于集合 T，一次“成功”的抽奖是指抽中了集合 T 中的任意一个灯泡。
单次抽奖成功的概率：在一次抽奖中，抽中 T 中任意一个灯泡的概率是所有 T 中灯泡的概率之和：

$$P_T = \sum_{c \in T} p_c = \sum_{c \in T} \frac{w_c}{W} = \frac{\sum_{c \in T} w_c}{W}$$

几何分布的期望：如果每次试验成功的概率是 P，那么第一次成功所需的期望试验次数是 $1/P$。
结论：因此，

$$E[\min_{c \in T} X_c] = \frac{1}{P_T} = \frac{1}{\frac{\sum_{c \in T} w_c}{W}} = \frac{W}{\sum_{c \in T} w_c}$$

将此代回容斥原理的公式：

$$E[\max_{c \in U_{S_{gifts}}} X_c] = \sum_{\emptyset \neq T \subseteq U_{S_{gifts}}} (-1)^{|T|-1} \frac{W}{\sum_{c \in T} w_c}$$

3. 动态规划优化计算
   直接枚举 $U_{S_{gifts}}$ 的所有子集 T 是不可行的，因为 $U_{S_{gifts}}$ 中最多有 $n^2 = 7^2 = 49$ 个灯泡，子集数量高达 2^{49}。

注意到 $\sum_{c\in T} w_c$ 的值域是 $[1, W]$，而 W 最大只有 1000。我们可以改变求和顺序，将对子集 T 的求和转换为对权重和 k 的求和：

$$E[\max_{c\in U_{S_{gifts}}} X_c] = \sum_{k=1}^{W} \left(\sum_{T\subseteq U_{S_{gifts}}, \sum_{c\in T} w_c = k} (-1)^{|T|-1} \right) \frac{W}{k}$$

我们可以将 $(-1)^{|T|-1}$ 拆成 $(-1)^{-1} \cdot (-1)^{|T|}$：

$$E[\max_{c\in U_{S_{gifts}}} X_c] = \sum_{k=1}^{W} (-1)^{-1} \left(\sum_{T\subseteq U_{S_{gifts}}, \sum_{c\in T} w_c = k} (-1)^{|T|} \right) \frac{W}{k}$$

$$E[\max_{c\in U_{S_{gifts}}} X_c] = \sum_{k=1}^{W} -f[k] \frac{W}{k}$$

其中，我们定义 $f[k]$ 为：

$$f[k] = \sum_{T\subseteq U_{S_{gifts}}, \sum_{c\in T} w_c = k} (-1)^{|T|}$$

这个 $f[k]$ 的定义非常关键。它实际上是多项式 $P(z) = \prod_{c\in U_{S_{gifts}}} (1 - z^{w_c})$ 中 z^k 的系数。

当 $T = \emptyset$ 时，$\sum_{c\in T} w_c = 0$，且 $|T| = 0$，所以 $f[0]$ 对应空集，其贡献为 $(-1)^0 = 1$。
对于每个灯泡 $c \in U_{S_{gifts}}$，它在乘积中贡献一个因子 $(1 - z^{w_c})$。
1 对应于不选择灯泡 c。
$-z^{w_c}$ 对应于选择灯泡 c，并将其权重 w_c 加入总和，同时贡献一个 (-1) 因子。
通过乘法展开，最终 z^k 的系数就是所有满足 $\sum_{c\in T} w_c = k$ 的子集 T 的 $(-1)^{|T|}$ 之和。
4. dfs + DP 实现细节
由于礼物线的总数 $m = 2n + 2$ 最大为 $2 \times 7 + 2 = 16$，所以 $2^m = 2^{16} = 65536$ 是一个可接受的范围。这意味着我们可以预处理所有 2^m 种查询的答案。

代码中的 dfs 函数就是用来实现这个预处理的。

dfs(line_idx, current_mask, current_f_array) 函数的含义：

line_idx: 当前正在考虑的礼物线的索引（从 0 到 m-1）。
current_mask: 一个位掩码，表示当前 DFS 路径中已经包含的礼物线集合。例如，如果 current_mask 的第 j 位是 1，则表示第 j 条礼物线被选中。

current_f_array: 这是一个 std::vector，大小为 W+1。它存储的是当前 current_mask 所对应的所有唯一灯泡集合 $U_{current_mask}$ 的多项式 $\prod_{c \in U_{current_mask}} (1 - z^{w_c})$ 的系数。即 current_f_array[k] 存储的是 z^k 的系数。
全局变量 vis 数组：

std::vector vis(n * n); 是一个全局数组（或者通过 lambda 捕获）。
vis[bulb_idx] 记录了灯泡 bulb_idx 在当前 DFS 路径中，被多少条已包含在 current_mask 中的礼物线所覆盖。
vis 数组的关键作用：它确保了每个灯泡 $c \in U_{S_{gifts}}$ 的因子 $(1 - z^{w_c})$ 只被应用到 current_f_array 中一次。当一个灯泡 x 第一次被 current_mask 包含时（即 vis[x] 从 0 变为 1），我们才将其贡献加入 current_f_array。
DFS 逻辑：

基准情况 (line_idx == m)：

这意味着我们已经考虑了所有 m 条礼物线（从 0 到 m-1）。
此时的 current_mask 代表了一个完整的、待查询的礼物集合 S_{gifts}。
current_f_array 已经正确地包含了 $U_{current_mask}$ 中所有灯泡的贡献，即它是多项式 $\prod_{c \in U_{current_mask}} (1 - z^{w_c})$ 的系数数组。
根据公式 $E[\max_{c \in U_{S_{gifts}}} X_c] = \sum_{k=1}^{W} -f[k] \frac{W}{k}$，计算最终答案：
Z res = 0;
for (int k = 1; k <= W; k++) {
res += current_f_array[k] * coef[k]; // coef[k] = W / k
}
ans[current_mask] = -res; // 存储结果
然后返回。
递归步骤： a. 不包含当前礼物线 line_idx： * 我们探索不包含第 line_idx 条礼物线的子问题。 * 调用 dfs(line_idx + 1, current_mask, current_f_array)。 * 重要：current_f_array 是按值传递的（std::vector 的拷贝），所以这里传递的是一个副本。这个副本在当前分支的后续操作中不会被修改，也不会影响到其他分支。

b. 包含当前礼物线 line_idx： * 我们探索包含第 line_idx 条礼物线的子问题。 * 更新 current_f_array： * 遍历第 line_idx 条礼物线上的所有灯泡 x（即 a[line_idx] 中的每个元素）。 * 对于每个灯泡 x： * 如果 vis[x] == 0：这表示灯泡 x 是第一次被当前 current_mask 包含（即，它是 $U_{current_mask}$ 中的一个新元素）。 * 将 vis[x] 增加 1。 * 现在，我们需要将灯泡 x 的贡献 $(1 - z^{w_x})$ 乘到当前的多项式 current_f_array 上。这相当于一个背包 DP 的过

程： 对于 j 从 W 递减到 w[x]： current_f_array[j] -= current_f_array[j - w[x]] 这个操作的含义是：新的 $f'[j]$ 等于旧的 $f[j]$ 减去旧的 $f[j-w_x]$。 这正是多项式乘法 $(A_0+A_1z+\dots)(1-z^k)$ 的系数更新规则： $(A_0+A_1z+\dots)(1-z^k)=(A_0+A_1z+\dots)-(A_0z^k+A_1z^{k+1}+\dots)$ 所以 z^j 的系数变为 A_j-A_{j-k}。 * 递归调用： * 调用 dfs(line_idx + 1, current_mask | (1 << line_idx), current_f_array)。 * current_mask | (1 << line_idx) 表示将第 line_idx 条礼物线加入到 current_mask 中。 * 回溯 vis 数组： * 在递归调用返回后，对于第 line_idx 条线上的每个灯泡 x，执行 vis[x]--。 * 这个回溯操作非常重要，它将 vis 数组恢复到进入当前 line_idx 状态之前的状态，以便其他 DFS 分支能够正确判断灯泡是否为新。 * 注意，current_f_array 是按值传递的，所以它的副本在递归返回后会被销毁，不需要手动“撤销”对 current_f_array 的修改。

初始调用：

在 solve() 函数中，首先初始化 f 数组：f[0] = 1，其余 f[k] = 0。这代表了空集合的多项式 $\prod_{c\in\emptyset}(1-z^{w_c})=1$。
然后调用 dfs(0, 0, f) 开始整个计算过程。
5. 示例说明 f[k] 的计算
假设我们有灯泡 c_1,c_2,c_3，权重分别为 w_1,w_2,w_3。 我们想计算 $P(z)=(1-z^{w_1})(1-z^{w_2})(1-z^{w_3})$ 的系数。

初始 f 数组：f[0]=1, 其他为 0。

处理 c_1 (权重 w_1)：

f[j] -= f[j - w_1] for j from W down to w_1.
结果：f[0]=1, f[w_1]=-1, 其他为 0。 (对应 $1-z^{w_1}$)
处理 c_2 (权重 w_2)：

f[j] -= f[j - w_2] for j from W down to w_2.
如果 $w_1\neq w_2$：
f[0] 保持 1。
f[w_1] 保持 -1。
f[w_2] 变为 -1 (因为 f[w_2] -= f[0] = 0 - 1 = -1)。
f[w_1+w_2] 变为 1 (因为 f[w_1+w_2] -= f[w_1] = 0 - (-1) = 1)。
结果：f[0]=1, f[w_1]=-1, f[w_2]=-1, f[w_1+w_2]=1，其他为 0。 (对应 $(1-z^{w_1})(1-z^{w_2})$)
处理 c_3 (权重 w_3)：

f[j] -= f[j - w_3] for j from W down to w_3.

以此类推，最终 f 数组将包含 $(1-z^{w_1})(1-z^{w_2})(1-z^{w_3})$ 的所有系数。
vis 数组的作用就是确保每个灯泡的贡献只被计算一次。例如，如果灯泡 x 既在礼物线 L_A 上，又在礼物线 L_B 上：

当 DFS 路径第一次包含 L_A 时，如果 x 之前不在任何已包含的线中 (vis[x]==0)，那么 x 的贡献会被加入 f 数组，并且 vis[x] 变为 1。
当 DFS 路径随后包含 L_B 时，即使 L_B 包含 x，由于此时 vis[x] 已经为 1，灯泡 x 的贡献不会再次被加入 f 数组。
当 DFS 回溯，不再包含 L_A 时，vis[x] 会减 1。如果 L_B 也不再包含，vis[x] 最终会回到 0。
这种 dfs 结合 vis 数组和背包 DP 的方式，能够高效且正确地计算出所有 2^m 种目标礼物组合对应的 f 数组，从而得到它们的期望值。